

Разбор миграций базы данных

Информационная система городской телефонной сети

Что делает каждый файл backend/migrations/0001...0014 — простыми словами

Содержание

1	Что такое миграции	2
2	0001 — типы (enum) и расширение	4
3	0002 — города, адреса и АТС	4
4	0003 — номера и абоненты	4
5	0004 — журнал звонков (CDR)	5
6	0005 — биллинг	5
7	0006 — очередь установки и таксофоны	6
8	0007 — пользователи и права (RBAC)	6
9	0008 — триггеры (целостность)	6
10	0009 — функция и представления	7
11	0010 — начальные данные (справочники)	7
12	0011 — права для оставшихся ресурсов	8
13	0012 — личный кабинет абонента	8
14	0013 — русские тексты триггеров (донакат)	9
15	0014 — русские описания ролей и прав (донакат)	9

1 Что такое миграции

Миграции — это пронумерованные SQL-файлы (0001_..., 0002_..., ... 0014_...), которые **по очереди** строят базу данных с нуля: создают типы, таблицы, связи, ограничения, триггеры, представления и заливают начальные данные. Порядок важен: нельзя сослаться на таблицу, которой ещё нет, поэтому, например, типы (0001) идут раньше таблиц, а таблицы — раньше триггеров.

Важно. Кто их применяет: в Docker — отдельный сервис migrate (прогоняет все файлы по порядку на свежей базе), локально — команда `make migrate`. Каждый файл выполняется один раз.

1.1 Базовый словарь (читать перед разбором)

Команды, которые меняют **структуру** БД, называют **DDL** (Data Definition Language). Вот всё, что встретится:

Конструкция	Что значит
CREATE TYPE x AS ENUM (...)	создать перечисление — тип, который принимает только значения из списка
CREATE TABLE t (...)	создать таблицу с колонками
CREATE INDEX ...	создать индекс — ускоряет поиск/фильтрацию по колонке
CREATE VIEW v AS SELECT ...	создать представление — сохранённый запрос как «виртуальная таблица»
CREATE TRIGGER ...	создать триггер — авто-проверку при изменении таблицы
INSERT INTO t ...	добавить строки (заливка начальных данных)
ALTER TABLE t ADD COLUMN ...	изменить уже существующую таблицу (добавить колонку)

Типы данных колонок:

Тип	Что хранит
BIGSERIAL	целое, которое само растёт (1, 2, 3, ...). Используется для id
BIGINT	большое целое (для ссылок на чужой id)
INTEGER / SMALLINT	целые поменьше (счётчики, год/месяц)
TEXT	строка любой длины
BOOLEAN	true / false
DATE	дата без времени
TIMESTAMPZ	дата + время с часовым поясом
NUMERIC(12,2)	точное десятичное число (для денег — никогда не float!): 12 знаков, 2 после запятой

Ограничения (правила на данные):

Ограничение	Что гарантирует
PRIMARY KEY	уникальный идентификатор строки (id)
REFERENCES t(id) (FOREIGN KEY)	внешний ключ — значение должно ссылаться на существующую строку в t
NOT NULL	поле обязательно
UNIQUE	значение не повторяется
DEFAULT x	значение по умолчанию, если не передано
CHECK (...)	условие, которому обязано удовлетворять значение

Что делать при удалении строки, на которую ссылаются (ON DELETE ...):

Действие	Поведение (пример)
CASCADE	удалить и зависимые строки (удалили АТС → удалась её строка-подтип)
RESTRICT	запретить удаление, пока есть ссылки (нельзя удалить номер, если на нём есть абонент)
SET NULL	обнулить ссылку (удалили город → у звонка dest_city_id станет NULL)

2 0001 — типы (enum) и расширение

Создаёт. Расширение btree_gist и 15 enum-типов: pbx_type, line_type, intercity_status, gender, subscriber_category и др.

```
CREATE TYPE line_type AS ENUM ('main', 'parallel', 'paired');
CREATE TYPE intercity_status AS ENUM ('none', 'open', 'closed');
```

- enum (перечисление) — это тип-«выпадающий список»: колонка такого типа примет только перечисленные значения. Например, тип линии может быть только main / parallel / paired, а не произвольная строка. Это первая линия защиты от мусора в данных.
- Типы создаются **раньше** таблиц, потому что таблицы будут на них ссылаться (колонка line_type line_type).
- CREATE EXTENSION btree_gist — подключает доп. возможности PostgreSQL (нужно для некоторых видов ограничений).

3 0002 — города, адреса и АТС

Создаёт. Таблицы city, address, pbx и три подтипа АТС: pbx_city, pbx_department, pbx_institution.

```
CREATE TABLE city (
    id          BIGSERIAL PRIMARY KEY,
    name        TEXT NOT NULL UNIQUE,
    is_home     BOOLEAN NOT NULL DEFAULT FALSE,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX uq_city_home ON city (is_home) WHERE is_home;
```

- id BIGSERIAL PRIMARY KEY — авто-номер строки.
- name TEXT NOT NULL UNIQUE — имя обязательно и не повторяется.
- is_home — флаг «свой город». Хитрый **частичный уникальный индекс** ... WHERE is_home гарантирует, что строк с is_home = true будет **не больше одной** (своих городов один).

Таблица pbx (АТС) хранит общее для всех типов: имя, код, район, ёмкость, каналы. А три таблицы-подтипа добавляют специфичные поля:

```
CREATE TABLE pbx_city (
    pbx_id BIGINT PRIMARY KEY REFERENCES pbx(id) ON DELETE CASCADE,
    intercity_enabled BOOLEAN NOT NULL DEFAULT TRUE,
    region_code TEXT
);
```

- Это **наследование** таблицами (class-table inheritance): pbx_city.pbx_id одновременно и первичный ключ, и ссылка на pbx(id). То есть строка-подтип «расширяет» строку pbx в отношении один-к-одному.
- ON DELETE CASCADE — удалили АТС → её строка-подтип удалится автоматически.
- CHECK (free_channels <= total_channels) в pbx — свободных каналов не может быть больше, чем всего.

4 0003 — номера и абоненты

Создаёт. Таблицы phone_number и subscriber — сердце предметной области.

```
CREATE TABLE phone_number (
    id          BIGSERIAL PRIMARY KEY,
    number      TEXT NOT NULL UNIQUE,
    pbx_id      BIGINT NOT NULL REFERENCES pbx(id) ON DELETE RESTRICT,
    line_type   line_type NOT NULL DEFAULT 'main',
    intercity   intercity_status NOT NULL DEFAULT 'none',
    status      number_status NOT NULL DEFAULT 'free',
```

```
address_id BIGINT REFERENCES address(id) ON DELETE RESTRICT, ...
);
```

- number UNIQUE — номер уникален.
- pbx_id ... REFERENCES pbx ON DELETE RESTRICT — каждый номер принадлежит АТС; нельзя удалить АТС, пока у неё есть номера.
- line_type, intercity, status — это enum-колонки из 0001, со значениями по умолчанию.

```
CREATE TABLE subscriber (
    ...
    birth_date DATE NOT NULL CHECK (birth_date > DATE '1900-01-01' AND birth_date <
CURRENT_DATE),
    category subscriber_category NOT NULL DEFAULT 'regular',
    privilege privilege_kind,
    phone_number_id BIGINT NOT NULL REFERENCES phone_number(id) ON DELETE RESTRICT,
    CONSTRAINT chk_privilege CHECK (
        (category = 'privileged' AND privilege IS NOT NULL) OR
        (category = 'regular' AND privilege IS NULL))
);
```

- birth_date ... CHECK (...) — дата рождения должна быть «в прошлом» и не раньше 1900 года.
- phone_number_id — абонент ссылается на номер. Несколько абонентов могут ссылаться на **один** номер (параллельный/спаренный телефон).
- CONSTRAINT chk_privilege — **важное правило**: если абонент льготник (category='privileged'), у него **обязан** быть указан вид льготы; если простой — вид льготы **не указывается**. Так данные не противоречат друг другу.
- Множество CREATE INDEX ниже — ускоряют типовые выборки (по фамилии, по категории, по номеру).

5 0004 — журнал звонков (CDR)

Создаёт. Таблицу call_record — все звонки (местные, внутренние, внешние, межгород).

```
CREATE TABLE call_record (
    id BIGSERIAL PRIMARY KEY,
    from_number_id BIGINT NOT NULL REFERENCES phone_number(id) ON DELETE CASCADE,
    call_type call_type NOT NULL,
    dest_city_id BIGINT REFERENCES city(id) ON DELETE SET NULL,
    dest_number_id BIGINT REFERENCES phone_number(id) ON DELETE SET NULL,
    started_at TIMESTAMPTZ NOT NULL,
    duration_sec INTEGER NOT NULL CHECK (duration_sec >= 0),
    cost NUMERIC(12,2) NOT NULL DEFAULT 0 CHECK (cost >= 0),
    CONSTRAINT chk_intercity_city CHECK (call_type <> 'intercity' OR dest_city_id IS NOT
NULL)
);
```

- from_number_id — с какого номера звонок (ON DELETE CASCADE — удалили номер → его звонки тоже удалятся).
- dest_city_id — город назначения (для межгорода), dest_number_id — вызываемый номер (для местных).
- cost NUMERIC(12,2) — стоимость деньгами (точный тип).
- chk_intercity_city — если звонок межгородний, у него **обязан** быть указан город (dest_city_id IS NOT NULL). Читается так: «либо тип не межгород, либо город задан».

6 0005 — биллинг

Создаёт. Таблицы tariff, billing_settings, invoice, payment, penalty, notification.

- **tariff** — стоимость абонплаты по (тип линии × наличие межгорода). UNIQUE (line_type, with_intercity, valid_from) — не больше одного тарифа на комбинацию.

- **billing_settings** — таблица-**одиночка** (id SMALLINT PRIMARY KEY DEFAULT 1 CHECK (id = 1) — единственная строка с id=1). Хранит скидку льготникам, пеню, день оплаты (20), отсрочку.
- **invoice** (счёт) — kind (абонплата/межгород), период (год+месяц), сумма, срок (due_date), статус. UNIQUE (subscriber_id, kind, period_year, period_month) — один счёт на абонента/вид/месяц (чтобы не было дублей за один период).
- **payment** (оплата), **penalty** (пеня), **notification** (письменное уведомление с дедлайном). У уведомления CHECK (deadline >= sent_at) — срок не раньше даты отправки.
- Везде ON DELETE CASCADE на subscriber_id: удалили абонента → удалились его счета/оплаты/пени.

7 0006 — очередь установки и таксофоны

Создаёт. Таблицы installation_queue (очередь на установку) и public_phone (таксофоны/общественные).

- **installation_queue** — заявка на установку: ФИО заявителя, тип очереди (regular/privileged — обычная/льготная), адрес, желаемая АТС, статус (waiting → installed), выделенный номер.
- **public_phone** — вид (public/payphone), АТС, адрес, флаг active.
- Внешние ключи ON DELETE RESTRICT на адрес/АТС — нельзя удалить адрес/АТС, пока на них ссылается заявка или таксофон.

8 0007 — пользователи и права (RBAC)

Создаёт. Таблицы app_user, role, permission, role_permission, user_role.

Это **ролевая система** (как ACL). Идея: пользователю назначают **роли**, роли содержат **права**.

```
CREATE TABLE permission (
  id BIGSERIAL PRIMARY KEY,
  code TEXT NOT NULL UNIQUE,    -- 'subscriber:read'
  description TEXT
);
CREATE TABLE role_permission (
  role_id BIGINT NOT NULL REFERENCES role(id) ON DELETE CASCADE,
  permission_id BIGINT NOT NULL REFERENCES permission(id) ON DELETE CASCADE,
  PRIMARY KEY (role_id, permission_id)
);
```

- app_user — оператор/админ. password_hash — пароль хранится не в открытом виде, а как Argon2-хеш.
- is_superuser — обходит проверки прав.
- permission.code — право вида сущность:действие (subscriber:read).
- **role_permission** и **user_role** — это **таблицы-связки** (многие-ко-многим): «роли ↔ права» и «пользователи ↔ роли». У них **составной первичный ключ** PRIMARY KEY (a, b) — пара не повторяется.
- Роли и права лежат **в БД**, поэтому суперадмин может всё перенастроить без правки кода (требование «роли не прибиты гвоздями»).

9 0008 — триггеры (целостность)

Создаёт. 8 функций (включая 2 для русских подписей enum) и 7 триггеров, которые автоматически проверяют сложные правила при изменении данных.

Триггер — функция, которая срабатывает **сама** при вставке/изменении строки. Нужен там, где простого CHECK мало (правило затрагивает другие строки/таблицы). BEFORE INSERT OR UPDATE — «перед» записью; NEW — новая строка; RAISE EXCEPTION — бросить ошибку (отменить операцию).

Пример — лимит абонентов на номер по типу линии:

```
CREATE FUNCTION trg_subscriber_count_check() RETURNS trigger AS $$
DECLARE lt line_type; cnt INTEGER; max_subs INTEGER;
BEGIN
```

```

SELECT line_type INTO lt FROM phone_number WHERE id = NEW.phone_number_id;
SELECT count(*) INTO cnt FROM subscriber
  WHERE phone_number_id = NEW.phone_number_id AND id <> NEW.id;
max_subs := CASE lt WHEN 'main' THEN 1 WHEN 'paired' THEN 2 WHEN 'parallel' THEN
2147483647 END;
IF cnt + 1 > max_subs THEN
  RAISE EXCEPTION 'К номеру № % (тип линии «%») можно подключить не более %
абонент(ов)', ...;
END IF;
RETURN NEW;
END; $$ LANGUAGE plpgsql;

```

Все 7 триггеров (что проверяют):

1. соответствие подтипа АТС её типу (×3 — на pbx_city/department/institution);
2. межгород только у городских АТС; у замкнутых сетей — none;
3. лимит абонентов на номер по типу линии (пример выше);
4. параллельные/спаренные абоненты — обязательно в одном доме;
5. авто-синхронизация статуса номера (free ↔ active) при появлении/удалении абонента.

Важно. Если запрос нарушит триггер, PostgreSQL отменит операцию и бросит ошибку, а сервер превратит её в ответ 400 с понятным текстом. Так данные остаются корректными при любом доступе к базе.

10 0009 — функция и представления

Создаёт. Функцию `fn_subscriber_monthly_fee` и представления `v_subscriber_full`, `v_subscriber_debt`, `v_pbx_stats`.

Представление (VIEW) — это сохранённый SELECT, к которому обращаются как к таблице. Нужно, чтобы не повторять одни и те же JOIN-ы в каждом запросе.

- `v_subscriber_full` — «плоский» абонент: абонент + его номер + АТС + адрес + вычисленный возраст. На нём стоят запросы 1, 3, 6, 7, 10, 13.
- `v_subscriber_debt` — долг абонента, разбитый на абонплату / межгород / пени (суммирует неоплаченные счета). Запросы 3, 4, 13.
- `v_pbx_stats` — по каждой АТС: число свободных номеров, всего номеров, абонентов.
- `fn_subscriber_monthly_fee(id)` — **функция** (мини-программа в БД): считает абонплату по тарифу и применяет скидку 50% льготнику.

Важно. Подробный строчный разбор этих представлений и всех 13 запросов — в отдельном PDF `docs/queries.pdf`.

11 0010 — начальные данные (справочники)

Создаёт. Каталог прав, системные роли, настройки биллинга и тарифы.

Здесь не структура, а **данные**, без которых приложение не работает. Первый блок — это маленькая программа на языке БД (`DO $$ ... $$`), которая в двух циклах создаёт права для всех сущностей × действий:

```

DO $$
DECLARE ent TEXT; act TEXT;
  ents TEXT[] := ARRAY['pbx', 'subscriber', ... , 'role'];
  acts TEXT[] := ARRAY['read', 'create', 'update', 'delete'];
BEGIN
  FOREACH ent IN ARRAY ents LOOP
    FOREACH act IN ARRAY acts LOOP
      INSERT INTO permission(code, description)
      VALUES (ent || ':' || act, ...) ON CONFLICT (code) DO NOTHING;
    
```

```

        END LOOP;
    END LOOP;
    ...
END $$;

```

- ent || ':' || act — склейка строк: получаются коды pbx:read, pbx:create, ... (15 сущностей × 4 действия = 60 прав) + спец-права analytics:read, raw_query:run, rbac:manage. Человекочитаемое описание к каждому праву собирается по-русски функциями perm_action_ru() / perm_entity_ru() (pbx:read → «Просмотр — АТС»).
- ON CONFLICT (code) DO NOTHING — если право уже есть, не падать (можно запускать повторно — **идемпотентность**).
- Далее создаются 3 роли (superadmin, operator, viewer) и им раздаются права через INSERT ... SELECT: суперадмину — все; viewer — все ...:read; operator — read/create/update, кроме управления пользователями/ролями.
- В конце — одна строка billing_settings и 6 тарифов.

12 0011 — права для оставшихся ресурсов

Создаёт. Права для подтипов АТС (pbx_city/department/institution) и настроек биллинга, и выдаёт их ролям.

Когда позже добавили CRUD для подтипов АТС и страницу настроек, понадобились новые права (pbx_city:read и т.д.). Этот файл их создаёт и раздаёт ролям тем же приёмом, что и 0010 (циклом + INSERT ... SELECT ... ON CONFLICT DO NOTHING).

13 0012 — личный кабинет абонента

Создаёт. Таблицу customer (аккаунт абонента) и добавляет связи в subscriber и installation_queue.

```

CREATE TABLE customer (
    id BIGSERIAL PRIMARY KEY,
    login TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    last_name TEXT NOT NULL, first_name TEXT NOT NULL, middle_name TEXT,
    gender gender NOT NULL, birth_date DATE NOT NULL CHECK (...),
    category subscriber_category NOT NULL DEFAULT 'regular',
    privilege privilege_kind,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    CONSTRAINT chk_customer_privilege CHECK (...)
);

```

```

ALTER TABLE subscriber      ADD COLUMN customer_id BIGINT REFERENCES customer(id) ON
DELETE SET NULL;
ALTER TABLE installation_queue ADD COLUMN customer_id BIGINT REFERENCES customer(id) ON
DELETE SET NULL;

```

- customer — отдельный аккаунт горожанина (логин/пароль + личные данные), не путать с app_user (это сотрудники).
- **ALTER TABLE ... ADD COLUMN** — изменяет **уже существующую** таблицу: добавляет в subscriber и installation_queue ссылку на аккаунт абонента. Так заявку и линию можно связать с тем, кто её подал.
- ON DELETE SET NULL — удалили аккаунт → у абонента/заявки ссылка просто обнулится (сама линия/заявка не пропадёт).

Важно. Эта миграция показывает, как развивать схему: новые возможности добавляют **новым** файлом-миграцией (ALTER/CREATE), не трогая старые. Старые миграции после применения не меняют.

14 0013 — русские тексты триггеров (донакат)

Создаёт. Переопределяет функции триггеров из 0008, чтобы сообщения об ошибках были на русском — **без** пересоздания базы.

Тексты ошибок из триггеров (RAISE EXCEPTION) видит оператор прямо на сайте, поэтому они должны быть понятными и на русском. Для **новых** баз тексты уже исправлены в 0008, но база, созданная **раньше**, это изменение «не увидит»: применённые миграции повторно не запускаются. Поэтому правка оформлена **отдельным** файлом.

```
CREATE OR REPLACE FUNCTION trg_number_intercity_check() RETURNS trigger AS $$
... RAISE EXCEPTION 'Для номера городской АТС % межгород должен быть открыт или
закрыт ...' ...
$$ LANGUAGE plpgsql;
```

- CREATE OR REPLACE FUNCTION — заменяет **тело** функции, не трогая таблицы и сами триггеры. Триггеры ссылаются на функцию по имени, поэтому после замены сразу работают по-новому.
- Файл **идемпотентен**: его можно накатить на уже работающую базу (psql -f ../0013_triggers_ru.sql) — он лишь обновит функции.
- Заодно добавлены две подписи-помощника pbx_type_ru() / line_type_ru(): переводят значения enum внутри сообщений (city → «городская», main → «основной»), чтобы в тексте не светила латиница.

Важно. Это **правильный приём** изменения уже применённой БД: не редактировать старый файл «задним числом», а добавить новый с CREATE OR REPLACE / UPDATE.

15 0014 — русские описания ролей и прав (донакат)

Создаёт. Переводит на русский описания ролей и прав, заведённые в 0010, в **уже существующей** базе.

В админке (раздел «Роли») у каждой роли показывается описание; в 0010 они были на английском («Full access to everything», «Manage telephone-network data, run analytics & raw queries»). Этот файл обновляет их и в новых, и в старых базах.

```
-- описание прав собирается из кода: 'pbx:read' → 'Просмотр — АТС'
UPDATE permission SET description =
    perm_action_ru(split_part(code, ':', 2)) || ' — ' || perm_entity_ru(split_part(code, ':',
1))
WHERE code LIKE '%:%' AND code NOT IN ('analytics:read', 'raw_query:run', 'rbac:manage');
UPDATE role SET description = 'Полный доступ ко всему' WHERE name = 'superadmin';
```

- UPDATE ... WHERE ... правит **существующие** строки — в отличие от INSERT ... ON CONFLICT DO NOTHING в 0010, который существующие строки не трогает (поэтому донакат и нужен).
- split_part(code, ':', 1/2) — разбирает код права на «сущность» и «действие».
- Тоже идемпотентен: повторный запуск просто перезапишет те же тексты.

См. также: схема БД — docs/schema.dbml, разбор запросов — docs/queries.pdf, общий разбор — docs/guide.pdf.